

Week 5: Lexical Analysis Implementation

Learning Objectives

By the end of this lab, students will be able to:

- ~~Draw state diagrams (NFA/DFA) for language tokens~~
- ~~Use JFLAP for automata visualization~~
- Implement lexical analyzers using multiple approaches
- Compare different implementation strategies

Theoretical Background

Reference: Dragon Book Chapter 3 (Lexical Analysis)

A lexical analyzer (scanner) reads the source program and groups characters into meaningful sequences called lexemes. For each lexeme, the scanner produces a token of the form: `<token-name, attribute-value>`.

Token Categories for a Simple Language

Token Type	Pattern	Examples
KEYWORD	Reserved words	if, else, while, int, float, return
IDENTIFIER	letter(letter digit)*	count, x1, myVar
INTEGER	digit+	42, 0, 123
FLOAT	digit+.digit+	3.14, 0.5
OPERATOR	+, *, /, =, <, >, ==, !=	+, ==, <=
DELIMITER	(,), {, }, ;, ,,	{, }, ;
STRING	"."	"hello", "world"
COMMENT	// or /* */	// comment, /* block */

Lab Tasks

Task 1: Implementation Approaches

Implement the lexical analyzer using THREE different approaches:

Approach 1: State-Based (Direct Coded)

```
function nextToken():
    state = START
    while true:
        char = getNextChar()
        switch(state):
            case START:
                if isLetter(char): state = IN_ID
                else if isDigit(char): state = IN_NUM
                else if char == '"': state = IN_STRING
                ...
            case IN_ID:
                if isAlphanumeric(char): continue
                else:
```

```

        ungetChar()
        return makeToken(ID, lexeme)
    ...

```

Approach 2: Stateless Approach

```

function nextToken():
    skipWhitespace()
    char = peekChar()

    if isLetter(char):
        return scanIdentifier()
    else if isDigit(char):
        return scanNumber()
    else if char == '"':
        return scanString()
    else:
        return scanOperator()

```

Approach 3: Transition Table Based

```

// Transition table: table[state][charClass] = nextState
transitionTable = [
    // LETTER DIGIT QUOTE PLUS ...
    [ 1,  2,  3,  4,  ...], // State 0
    [ 1,  1, -1, -1,  ...], // State 1 (ID)
    [-1,  2, -1, -1,  ...], // State 2 (NUM)
    ...
]

```

```

acceptingStates = {1: ID, 2: NUMBER, ...}

```

```

function nextToken():
    state = 0
    while state != ERROR:
        char = getNextChar()
        charClass = classifyChar(char)
        state = transitionTable[state][charClass]
    return makeToken(acceptingStates[prevState], lexeme)

```

Bonus: Compressed Table Approach

Implement table compression to reduce memory usage for sparse transition tables.

Input Specification

Input: Source code file in a simple programming language

Sample Input (program.microjava):

```

// Sample program
int main() {
    int x = 10;
    float pi = 3.14159;

    if (x > 5) {
        x = x + 1;
    }
}

```

```

}

/* Multi-line
   comment */

string msg = "Hello, World!";
return 0;
}

```

Output Specification

Output: List of tokens with their types and values

Sample Output:

Token Stream:

Line	Token Type	Lexeme	Value
1	COMMENT	// Sample...	-
2	KEYWORD	int	-
2	IDENTIFIER	main	-
2	LPAREN	(	-
2	RPAREN	)	-
2	LBRACE	{	-
3	KEYWORD	int	-
3	IDENTIFIER	x	-
3	ASSIGN	=	-
3	INTEGER	10	10
3	SEMICOLON	;	-
4	KEYWORD	float	-
4	IDENTIFIER	pi	-
4	ASSIGN	=	-
4	FLOAT	3.14159	3.14159
4	SEMICOLON	;	-
...			
Total Tokens: 45			
Lexical Errors: 0			

Deliverables

- ~~1. State diagrams (hand-drawn and JFLAP files)~~
2. Source code for all three implementation approaches
3. Comparison report discussing pros/cons of each approach
4. Test suite with various input programs

Evaluation Criteria

Criteria	Marks
State diagram correctness (NFA/DFA)	20%
State-based implementation	25%
Stateless implementation	20%
Transition table implementation	25%

Compressed table (Bonus)

10%